

LEOPARDO RH

Plateforme SaaS de Gestion RH · Marché MENA & Méditerranée

Module Surveillance Caméras

Do

Statut :	Planification active · Démarrage après MVP Phase 1
Effort :	~30,75 jours-développeur · 6 semaines · 1 développeur
Sprints :	5 sprints (A → E) + Roadmap post-phase documentée
Tâches :	60+ tâches détaillées avec priorités et dépendances
Stack :	Laravel 11 · Flutter · MediaMTX · WebRTC · PostgreSQL
Cible :	Plans Business (4 cam.) & Enterprise (illimité)

se

TABLE DES MATIÈRES

1.	Résumé Exécutif	1
2.	L'Opportunité Marché	2
2.1	Contexte et genèse	2
2.2	Proposition de valeur	2
2.3	Alignement architecture existante	2
2.4	Impact modèle économique	2
2.5	Analyse concurrentielle	3
2.6	Analyse risques	3
3.	Architecture Technique	4
3.1	Vue d'ensemble	4
3.2	Composants et responsabilités	4
3.3	Flux d'authentification sécurisé	4
3.4	Choix technologiques justifiés	5
3.5	Gestion réseau client (NAT traversal)	5
3.6	Scalabilité et performance	5
4.	Modèle de Données Complet	6
4.1	Nouvelles tables (schéma tenant)	6
4.2	Modifications tables existantes	7
4.3	Index et optimisations	7
4.4	Sécurité des données	7
5.	RBAC — Matrice des Permissions	8
6.	Contrats API Complets	9
6.1	Endpoints Laravel API V1	9
6.2	Endpoint interne MediaMTX	9
6.3	Schemas de requêtes/réponses	10
6.4	Gestion des erreurs	10
7.	Configuration MediaMTX	11
8.	Plan d'Implémentation — Toutes les Tâches	12
Sprint A	Infrastructure & Base de données	12
Sprint B	API Backend Laravel	13
Sprint C	Dashboard Web (Blade)	14
Sprint D	Application Flutter	15

Sprint E	Tests, Sécurité & Déploiement	16
9.	Tests — Stratégie Complète	17
10.	Déploiement et DevOps	18
11.	Documentation & Légal	19
12.	Roadmap Post Phase 1 Prime	20
12.1	Phase 2 — Enregistrement & Stockage	20
12.2	Phase 3 — Détection de Mouvement & IA	20
12.3	Phase 4 — Analytics Vidéo Avancé	21
12.4	Phase 5 — Intégration Ecosystem	21
13.	Récapitulatif et Definition of Done	22

1

RÉSUMÉ EXÉCUTIF

Vue d'ensemble du module pour le développeur assigné

Ce document constitue la spécification technique complète du **Module Surveillance Caméras** de Leopardo RH. Il est conçu pour permettre à un développeur senior de comprendre, concevoir et implémenter le module de façon autonome, sans nécessiter de réunions de clarification supplémentaires.

Objectif produit : permettre au manager d'une PME de visualiser ses caméras de surveillance IP directement depuis l'app mobile Leopardo RH — la même app depuis laquelle il suit les pointages de ses employés. Le tout avec gestion d'accès tiers (partage sécurisé à un associé, assureur ou partenaire sans création de compte).

Aspect	Détail
Stack vidéo	MediaMTX (proxy RTSP→WebRTC) + flutter_webrtc
Auth	Sanctum existant + stream_tokens JWT signés (TTL=4h)
Base de données	3 nouvelles tables dans le schéma tenant existant
Plans concernés	Business (4 caméras max) · Enterprise (illimité)
Effort total	~30,75 jours-développeur · 5 sprints · 6 semaines
Dépendance critique	MVP Phase 1 core doit être déployé et stable
Compatibilité cam.	Toute caméra IP supportant le protocole RTSP (>95% du marché)
Isolation données	Global Scope company_id existant couvre automatiquement les caméras

2 L'OPPORTUNITÉ MARCHÉ

2.1 — Contexte et genèse de l'idée

Leopardo RH permet déjà au manager de suivre ses employés en temps réel : présences, retards, absences. C'est une première couche de surveillance opérationnelle qui répond à la question *"Qui est là ?"*. La question complémentaire naturelle est *"Que se passe-t-il sur mon site ?"* — et c'est exactement ce que le module caméras adresse.

Les caméras de surveillance IP sont présentes dans la quasi-totalité des PME de notre marché cible (boutiques, entrepôts, ateliers, restaurants, chantiers). Or elles restent systématiquement dans une application séparée — NVR local avec interface datée, app propriétaire de la marque — que le manager consulte depuis un second appareil ou un second onglet. Cette friction est inutile.

2.2 — Proposition de valeur

"Un seul écran pour savoir qui est là ET voir ce qui se passe — sans jongler entre deux applications." Le manager ouvre Leopardo RH, voit ses pointages, puis tape sur l'icône Caméras et voit ses flux vidéo. Même app, même contexte, même authentification.

Trois dimensions de valeur :

- **Gain de temps** : plus besoin de passer d'une app à l'autre. En incident, le manager croise immédiatement pointage et vidéo.
- **Sécurité renforcée** : couplé aux alertes de présence existantes (arrivée tardive, absence), la vidéo apporte une preuve visuelle immédiate.
- **Délégation sécurisée** : le manager génère un lien temporaire pour un tiers (associé, assureur, agent de sécurité) sans créer de compte. Accès révoquant à tout moment.

2.3 — Alignement avec l'architecture existante

Cette feature s'insère sans remettre en cause aucun choix architectural existant :

Composant existant	Rôle dans le module caméras	Modification requise
Sanctum (auth tokens)	Valide l'accès aux caméras exactement comme il valide l'accès aux employés	Aucune
RBAC (rôles manager/employé)	Contrôle qui peut voir quelles caméras. Permission camera_view ajoutée.	Ajout permission

Composant existant	Rôle dans le module caméras	Modification requise
Multitenancy (company_id Global Scope)	Isole les caméras par entreprise — automatique	Aucune
PostgreSQL (schéma tenant)	Stocke les métadonnées et tokens d'accès caméras	3 nouvelles tables
plans.features JSONB	Active/désactive le module selon le plan souscrit	Ajout clé cameras
App Flutter	Affiche les flux via WebRTC (package flutter_webrtc)	Nouvel écran
Dashboard Blade	Interface d'administration des caméras	Nouvelles routes/vues

2.4 — Impact sur le modèle économique

Le module caméras constitue un levier de différenciation tarifaire fort, justifiant le positionnement exclusif Business/Entreprise et réduisant le churn en augmentant la valeur perçue de la plateforme.

Plan	Prix mensuel	Accès caméras	Limite	Impact attendu
Trial	Gratuit	Non	—	Incitation à upgrade
Starter	29 €/mois	Non	—	Argument de vente vers Business
Business	79 €/mois	Oui ✓	4 caméras	Rétention + upsell
Entreprise	199 €/mois	Oui ✓	Illimité	Rétention forte

2.5 — Analyse concurrentielle

Aucun concurrent direct sur notre marché cible (MENA, Maghreb, Méditerranée) n'intègre la surveillance vidéo dans une plateforme RH mobile. C'est une opportunité de différenciation unique.

Concurrent	RH Mobile	Caméras intégrées	Marché cible	Notre avantage
Sage HR	Partielle	Non	Europe	Mobile-first + caméras + prix PME
Paychex Flex	Oui	Non	USA	Couverture MENA + arabe
NVR constructeurs (Hikvision...)	Non	Oui (natif)	Global	Intégration RH + pointage couplé
Milestone XProtect	Non	Oui (avancé)	Enterprise	Prix PME + simplicité + mobile
Apps caméra standalone	Non	Partielle	Global	Contexte RH + auth unifiée

2.6 — Analyse risques et mitigations

Risque	Probabilité	Impact	Mitigation
Caméra non compatible RTSP	Faible	Moyen	95%+ caméras IP supportent RTSP. Guide modèles testés.
Latence trop haute sur 3G/4G	Moyenne	Élevé	Qualité adaptative (bitrate réduit). Option flux 360p.
Surcharge VPS si N flux simultanés	Faible	Élevé	MediaMTX passthrough (pas de transcodage). Limite max_cameras/plan.
NAT/firewall bloquant le flux RTSP	Haute	Élevé	Guide redirection de port. Option tunnel local (agent).

Risque	Probabilité	Impact	Mitigation
Réticence RGPD (FR, TN)	Haute	Élevé	Mention légale obligatoire à la 1re caméra. Opt-in explicite.
Token stream_token intercepté	Très faible	Élevé	TTL court (4h), TLS obligatoire, rotation auto.
Caméra exposée sans auth si MediaMTX mal configuré	Faible	Critique	Auth webhook obligatoire dans mediamtx.yml. Tests de sécurité.

3

ARCHITECTURE TECHNIQUE

Conception complète du système de streaming vidéo

3.1 — Vue d'ensemble du flux

Principe fondamental : Laravel ne touche jamais les flux vidéo. Il gère uniquement l'authentification et les permissions. Le flux vidéo brut est entièrement géré par MediaMTX. Laravel est dans le chemin d'autorisation mais jamais dans le chemin des données vidéo.

Le flux complet en 5 étapes :

- 1. Caméra IP → MediaMTX** : La caméra du client émet un flux RTSP continu. MediaMTX le reçoit en pull (se connecte à l'URL rtsp:// configurée) ou en push (la caméra envoie vers MediaMTX). Le flux est stocké en mémoire tampon, jamais sur disque en Phase 1 Prime.
- 2. App → Laravel (liste + tokens)** : L'app Flutter envoie GET /api/v1/cameras avec le token Sanctum habituel. Laravel vérifie le rôle, le plan tarifaire, génère un stream_token JWT signé (TTL=4h) pour chaque caméra autorisée et retourne la liste.
- 3. App → MediaMTX (ouverture flux WebRTC)** : L'app construit l'URL wss://proxy.leopardo-rh.com/cam/{id}/webrtc?token={stream_token} et initie une négociation WebRTC (SDP offer/answer via HTTP).
- 4. MediaMTX → Laravel (validation token)** : Avant d'autoriser le flux, MediaMTX appelle GET /internal/camera-token/verify avec le stream_token. Laravel valide la signature, l'expiration, et retourne allowed: true/false. Cette vérification est cacheable (Redis, TTL=30s) pour réduire la charge.
- 5. Flux vidéo direct (bypass Laravel)** : Une fois autorisé, le flux WebRTC circule directement entre MediaMTX et le client. Laravel n'est plus dans le chemin critique. Latence cible < 500ms sur WiFi.

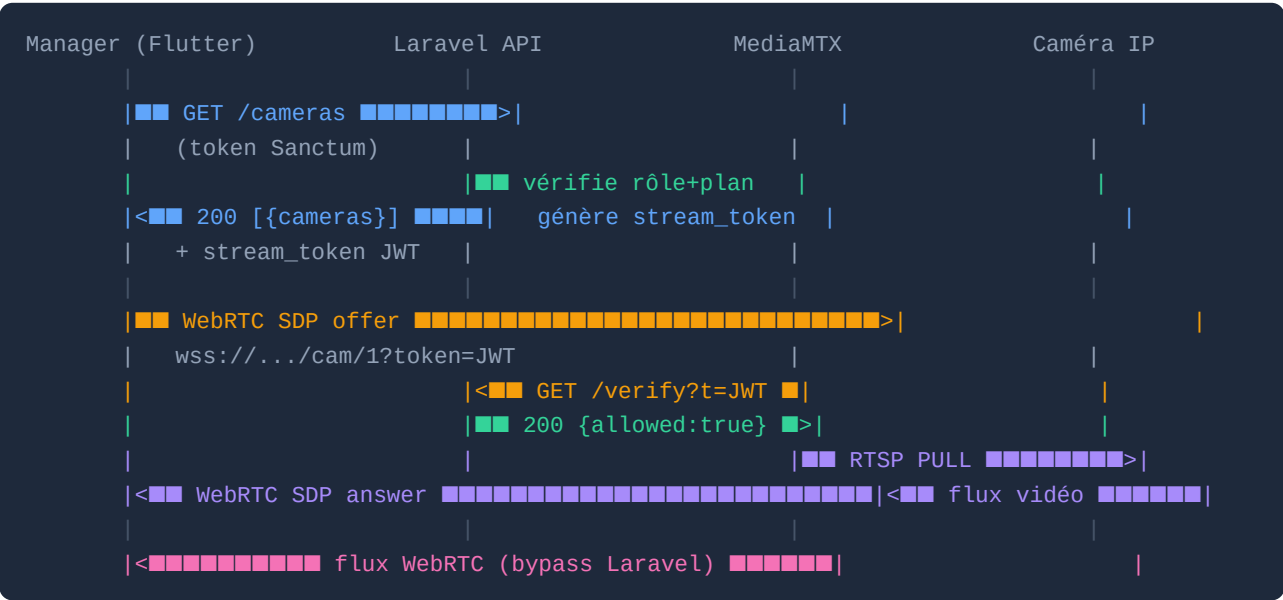
3.2 — Composants et responsabilités

Composant	Technologie	Rôle	Déployé où	Requis Phase 1P
Caméra IP	N'importe quelle cam RTSP	Source du flux vidéo	Chez le client	Oui
MediaMTX	Binaire Go (médiامتخ)	Proxy RTSP→WebRTC. Auth webhook. TLS.	VPS / Render Worker	Oui
TURN Server	coturn (open source)	NAT traversal pour clients derrière NAT	VPS (port UDP 3478)	Recommandé
Laravel API	Laravel 11 existant	Auth, CRUD cameras, génération tokens	Render (existant)	Oui

Composant	Technologie	Rôle	Déployé où	Requis Phase 1P
PostgreSQL	PostgreSQL 16 existant	Métadonnées caméras, tokens, permissions	Neon.tech (existant)	Oui
Redis (optionnel)	Redis	Cache validation tokens (performance)	Render / Upstash	Optionnel
App Flutter	Flutter + flutter_webrtc	Affichage flux, gestion accès tiers	iOS / Android	Oui
Dashboard Blade	Blade + Alpine.js	Admin caméras, formulaires, viewer web	Web (existant)	Oui

3.3 — Flux d'authentification sécurisé (diagramme)

Séquence complète pour un accès légitime au flux d'une caméra :



Important : une fois le flux WebRTC établi, Laravel n'intervient plus. La vérification du token par MediaMTX se fait en arrière-plan toutes les 30s pour détecter une révocation. En cas de révocation, MediaMTX coupe le flux dans les 30 secondes.

3.4 — Choix technologiques justifiés

Solution	RTSP→WebRTC	Auth token	Open source	Charge serveur	Complexité déploiement	Verdict
MediaMTX	Oui	Webhook	Oui	Très faible (Go, passthrough)	Simple (1 binaire)	✓ Re te nu
Janus Gateway	Oui	Plugin	Oui	Moyenne (C, transcodage)	Complexe (plugins)	Tr op co m ple xe
Wowza Streaming	Oui	Oui	Non	Faible	Simple	Pa ya nt (\$ \$ \$)
Nginx RTMP	HLS seulement	Non natif	Oui	Faible	Moyenne	La te nc e 10 s+
WebRTC P2P direct	Oui	Manuel	N/A	Nulle	Très complexe	Inf ais abl e N AT
GStreamer pipeline	Oui	Non natif	Oui	Haute (transcodage)	Très complexe	Tr op lou rd

3.5 — Gestion du réseau client (NAT traversal)

Le principal défi réseau est que les caméras se trouvent sur le réseau local du client, derrière un routeur NAT. Trois modes sont supportés, du plus simple au plus universel :

- **Mode A — Redirection de port (recommandé)** : Le client configure son routeur pour exposer le port RTSP de la caméra sur Internet. MediaMTX se connecte directement à l'IP publique + port. Guide illustré fourni par marque de routeur (Livebox, Box SFR, Freebox, routeurs Cisco...). Latence :

excellente (<200ms).

- **Mode B — TURN server** : Pour les clients derrière un NAT symétrique (entreprises avec firewall strict), un serveur TURN (coturn) déployé sur le VPS relaye les paquets WebRTC. MediaMTX + flutter_webrtc sont configurés avec l'URL TURN. Latence : bonne (<400ms). Coût : bande passante additionnelle.
- **Mode C — Agent local tunnel (Phase 2)** : Pour les cas impossibles (aucun accès routeur), un petit agent (binaire Go ou script Python) installé sur un PC du site crée un tunnel HTTPS persistant vers notre infrastructure. MediaMTX se connecte au tunnel. Universel mais nécessite une installation chez le client.

3.6 — Scalabilité et performance

Objectifs de performance Phase 1 Prime :

Métrique	Objectif Phase 1P	Mesure	Action si dépassé
Latence flux (WiFi)	< 500ms	Tests k6 + ffplay	Optimiser buffer MediaMTX
Latence flux (4G)	< 1500ms	Tests terrain	Réduire résolution par défaut
Flux simultanés / VPS	> 20 flux	Test charge k6	Scale horizontale MediaMTX
Temps validation token	< 100ms	Benchmark endpoint /verify	Activer cache Redis
Connexion flux initial	< 3s	Tests E2E Flutter	Optimiser SDP offer/answer
Disponibilité MediaMTX	> 99%	Monitoring Uptime Robot	Restart automatique + alerting

4

MODÈLE DE DONNÉES COMPLET

3 nouvelles tables + modifications tables existantes

4.1 — Table : cameras

Table principale stockant les métadonnées de chaque caméra. Les URLs RTSP sont chiffrées en base.

Colonne	Type	Contraintes	Description
id	INT	PK AUTO INCREMENT	Identifiant interne
company_id	UUID	FK → companies.id · NOT NULL · INDEX	Isolation multitenancy — Global Scope
name	VARCHAR(100)	NOT NULL	Nom affiché : "Entrée principale", "Entrepôt A"
rtsp_url	TEXT	NOT NULL · CHIFFRÉ AES-256	URL complète : rtsp://user:pass@ip:554/stream1
location	VARCHAR(200)	NULL	Description physique : "Hall RDC côté rue"
is_active	BOOLEAN	NOT NULL DEFAULT true	Soft-disable sans suppression
thumbnail_path	VARCHAR(255)	NULL	Capture statique S3/local (refresh toutes les 5min Phase 2)
sort_order	SMALLINT	NOT NULL DEFAULT 0	Ordre d'affichage dans la grille app
created_by	INT	FK → employees.id · NOT NULL	Manager ayant configuré la caméra
stream_path_override	VARCHAR(100)	NULL	Override du path MediaMTX si besoin (avancé)
metadata	JSONB	NULL DEFAULT '{}'	Extensible : marque, modèle, résolution déclarée
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
updated_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	
deleted_at	TIMESTAMPTZ	NULL	Soft delete (SoftDeletes Laravel)

4.2 — Table : camera_access_tokens

Gère les accès temporaires délégués à des tiers externes (sans compte) ou à des utilisateurs internes pour un accès hors-app.

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
company_id	UUID	FK · NOT NULL · INDEX	
camera_id	INT	FK → cameras.id · NOT NULL · INDEX	
token	VARCHAR(64)	UNIQUE · NOT NULL	random_bytes(32) → bin2hex. Impossible à deviner.
label	VARCHAR(150)	NULL	"Accès assureur AXA" — traçabilité interne
granted_to_email	VARCHAR(150)	NULL	Email tiers (informatif, aucun compte créé)
granted_to_name	VARCHAR(100)	NULL	Nom tiers affiché dans le dashboard
granted_by	INT	FK → employees.id · NOT NULL	Manager ayant généré le token
permissions	JSONB	NOT NULL DEFAULT '{"view":true}'	Extensible : view, ptz, audio
expires_at	TIMESTAMPTZ	NOT NULL	Expiration configurable : 1h, 6h, 24h, 7j, 30j
last_used_at	TIMESTAMPTZ	NULL	MAJ à chaque vérification réussie
use_count	INT	NOT NULL DEFAULT 0	Compteur d'accès pour audit
is_revoked	BOOLEAN	NOT NULL DEFAULT false	Révocation immédiate possible
ip_whitelist	JSONB	NULL	Phase 2 : liste d'IPs autorisées
created_at	TIMESTAMPTZ	NOT NULL DEFAULT NOW()	

4.3 — Table : camera_permissions

Gère les droits d'accès des utilisateurs internes (managers, superviseurs) sur des caméras spécifiques. Permet un accès granulaire : un superviseur ne voit que les caméras de sa zone.

Colonne	Type	Contraintes	Description
id	INT	PK AUTO	
company_id	UUID	FK · NOT NULL	
camera_id	INT	FK → cameras.id · NOT NULL	
employee_id	INT	FK → employees.id · NOT NULL	Utilisateur interne autorisé
can_view	BOOLEAN	NOT NULL DEFAULT true	Peut visualiser le flux en direct
can_share	BOOLEAN	NOT NULL DEFAULT false	Peut générer des accès tiers
can_manage	BOOLEAN	NOT NULL DEFAULT false	Peut modifier les paramètres de la caméra
granted_by	INT	FK → employees.id · NOT NULL	Manager ayant accordé la permission
granted_at	TIMESTAMPZ	NOT NULL DEFAULT NOW()	
expires_at	TIMESTAMPZ	NULL	Expiration optionnelle de la permission

Contrainte UNIQUE : (camera_id, employee_id) — une seule ligne de permission par couple.

4.4 — Modifications tables existantes

Table	Modification	SQL / Détail
plans	Ajouter cameras (bool) + max_cameras (int) dans features JSONB	UPDATE plans SET features = features '{"cameras":true,"max_cameras":4}' WHERE name='Business'
plans	max_cameras NULL pour Enterprise (illimité)	UPDATE plans SET features = features '{"cameras":true,"max_cameras":null}' WHERE name='Enterprise'
companies	Aucune modification	Le company_id suffit pour l'isolation
employees	Aucune modification	Les relations passent par camera_permissions

4.5 — Index et optimisations

```
-- Index critiques pour performance
CREATE INDEX idx_cameras_company ON cameras(company_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_cam_tokens_lookup ON camera_access_tokens(token, is_revoked, expires_at);
CREATE INDEX idx_cam_tokens_camera ON camera_access_tokens(camera_id, is_revoked);
CREATE UNIQUE INDEX idx_cam_perms_unique ON camera_permissions(camera_id, employee_id);
CREATE INDEX idx_cam_perms_employee ON camera_permissions(employee_id, company_id);

-- Validation token (requête la plus fréquente - appelée par MediaMTX)
SELECT 1 FROM camera_access_tokens
  WHERE token = $1 AND is_revoked = false AND expires_at > NOW();
-- Exécution < 1ms avec l'index idx_cam_tokens_lookup
```


5 RBAC — MATRICE DES PERMISSIONS

Le module caméras s'intègre dans le RBAC existant de Leopard RH sans modifier son architecture. La permission `camera_view` est vérifiée en combinaison avec le plan tarifaire (`features.cameras=true`).

Action	Super Admin	Manager Principal	Manager RH	Dept / Superviseur	Employé
Voir liste des caméras	✓ (toutes)	✓ (sa company)	✓ (sa company)	Ses zones*	✗
Visualiser flux en direct	✓	✓	✓	Ses zones*	✗
Voir grille multi-caméras	✓	✓	✓	Ses zones*	✗
Ajouter une caméra	✓	✓	✗	✗	✗
Modifier config caméra	✓	✓	✗	✗	✗
Désactiver / Supprimer cam.	✓	✓	✗	✗	✗
Tester connexion RTSP	✓	✓	✗	✗	✗
Générer accès tiers (lien)	✓	✓	✓	✗	✗
Voir accès tiers actifs	✓	✓	✓	✗	✗
Révoquer accès tiers	✓	✓	✓	✗	✗
Gérer permissions internes	✓	✓	✗	✗	✗
Voir historique d'accès	✓	✓	✓	✗	✗
Config. alertes mouvement (Ph.2)	✓	✓	✗	✗	✗
Voir recordings (Phase 2)	✓	✓	✓	Ses zones*	✗

* Zones = caméras listées dans `camera_permissions` pour cet `employee_id`. La logique de filtrage est dans `CameraPolicy::viewAny()` qui applique le scope additionnel si `manager_role` $\notin$ ['principal', 'rh'].

Phase 1 Prime : activer uniquement les rôles Principal et RH. Les droits Dept/Superviseur sont spécifiés dans le schéma `camera_permissions` mais le middleware vérifie `manager_role` pour les restreindre. Ils seront activés en Phase 2 après retours terrain.

6

CONTRATS API COMPLETS

Tous les endpoints avec schemas, exemples et codes d'erreur

6.1 — Endpoints Laravel API V1

Méthode	Endpoint	Auth	Rôle minimum	Description
GET	/api/v1/cameras	Sanctum	manager	Liste caméras + stream_tokens générés
POST	/api/v1/cameras	Sanctum	manager:principal	Crée nouvelle caméra (vérifie max_cameras)
GET	/api/v1/cameras/{id}	Sanctum	manager	Détail d'une caméra + token frais
PUT	/api/v1/cameras/{id}	Sanctum	manager:principal	Met à jour nom, URL, localisation
DELETE	/api/v1/cameras/{id}	Sanctum	manager:principal	Soft delete (is_active=false + deleted_at)
POST	/api/v1/cameras/test-rtsp	Sanctum	manager:principal	Teste connexion RTSP (ffprobe, timeout=5s)
GET	/api/v1/cameras/{id}/stream-token	Sanctum	manager	Refresh stream_token (appelé 30min avant expiration)
GET	/api/v1/cameras/{id}/access-tokens	Sanctum	manager	Liste tokens tiers actifs pour cette caméra
POST	/api/v1/cameras/{id}/access-tokens	Sanctum	manager	Crée token d'accès tiers
DELETE	/api/v1/cameras/{id}/access-tokens/{tid}	Sanctum	manager	Révoque un token tiers
GET	/api/v1/cameras/{id}/access-logs	Sanctum	manager	Historique accès (paginé, 30 jours)
GET	/internal/camera-token/verify	Secret Header	Interne (MediaMTX)	Vérifie validité stream_token ou access_token
GET	/view/cam	Token URL (?t=)	Public (avec token valide)	Page viewer tiers sans compte

6.2 — Schema : GET /api/v1/cameras (réponse)

```
{
  "data": [
    {
      "id": 1,
      "name": "Entrée principale",
      "location": "Hall RDC côté rue",
      "is_active": true,
      "sort_order": 0,
      "thumbnail_url": "https://cdn.leopardo-rh.com/.../thumb_1.jpg",
      "stream_url": "wss://proxy.leopardo-rh.com/cam/1/webrtc",
      "stream_token": "eyJhbGciOiJIUzI1NiJ9.eyJjYW0iOiEsImV4cCI6...\"",
      "token_expires_at": "2026-04-19T20:00:00Z",
      "created_at": "2026-03-01T10:00:00Z"
    }
  ],
  "plan_limit": { "max_cameras": 4, "current_count": 1 }
}
```

6.3 — Schema : POST /api/v1/cameras (corps de requête)

```
{
  "name": "Entrepôt B",           // required|string|max:100
  "rtsp_url": "rtsp://admin:pass@192.168.1.100:554/stream1", // required|url_rtsp
  "location": "Bâtiment B, aile nord", // nullable|string|max:200
  "sort_order": 1,               // nullable|integer|min:0
  "metadata": {                  // nullable|json
    "brand": "Hikvision",
    "model": "DS-2CD2143G2-I",
    "resolution": "2688x1520"
  }
}

// Validations :
// rtsp_url : regex /^rtsp:\\\\/.+/ + longueur max 1000
// Interdit : rtsp_url contenant des caractères de contrôle
// Vérifie max_cameras du plan avant création (403 si atteint)
```

6.4 — Schema : endpoint interne /internal/camera-token/verify

```
// Appelé par MediaMTX à chaque nouvelle connexion WebRTC
// Protégé par header : Authorization: Bearer {MEDIAMTX_INTERNAL_SECRET}

GET /internal/camera-token/verify
?token=eyJhbGciOiJIUzI1NiJ9... // stream_token JWT ou access_token tiers
&camera_id=1 // ID de la caméra demandée
&client_ip=185.x.x.x // IP du client (pour logs)

// Réponse 200 – accès autorisé :
{ "allowed": true, "company_id": "uuid-xxx", "type": "stream_token" }

// Réponse 200 – accès refusé (toujours 200 pour MediaMTX, flag dans le body) :
{ "allowed": false, "reason": "token_expired" }
// reasons: token_expired | token_revoked | camera_not_found | company_suspended
```

6.5 — Gestion des erreurs API

Code HTTP	Erreur	Cause	Message retourné
403	Forbidden	Plan ne supporte pas le module caméras	"Your plan does not include the cameras module. Upgrade to Business."
403	Forbidden	Limite max_cameras atteinte	"Camera limit reached for your plan (4 max). Upgrade to Enterprise."
403	Forbidden	Rôle insuffisant	"Insufficient role to manage cameras."
404	Not Found	Caméra d'une autre company ou supprimée	"Camera not found."
422	Unprocessable	URL RTSP invalide	"The rtsp_url must be a valid RTSP URL starting with rtsp://"
503	Service Unavailable	MediaMTX inaccessible lors du test RTSP	"Video proxy unavailable. Please try again."
408	Request Timeout	Test RTSP timeout après 5s	"Connection to camera timed out. Verify the URL and network."

7

CONFIGURATION MEDIAMTX

Fichier mediamtx.yml annoté et complet

MediaMTX est configuré pour : auth via webhook Laravel, TLS obligatoire, WebRTC activé, logs structurés, limites de connexions. Ce fichier est la référence de production.

```
# mediamtx.yml – Production Leopard RH
# Dernière révision : Avril 2026

# ■■ Authentification (CRITIQUE – sans ceci toute cam est publique)
authMethod: http
authHTTPAddress: https://api.leopardo-rh.com/internal/camera-token/verify
authHTTPExclude: [] # Aucune exclusion : toutes les cams nécessitent auth

# ■■ Réseau
rtspAddress: :8554
rtmpAddress: :1935 # RTMP désactivé en production
hlsAddress: :8888 # HLS désactivé – trop de latence
webrtcAddress: :8889

# ■■ TLS (obligatoire en production)
tlsEncryption: yes
tlsCert: /etc/letsencrypt/live/proxy.leopardo-rh.com/fullchain.pem
tlsKey: /etc/letsencrypt/live/proxy.leopardo-rh.com/privkey.pem

# ■■ WebRTC (ICE servers pour NAT traversal)
webrtcICEServers2:
  - url: stun:stun.l.google.com:19302
  - url: turn:turn.leopardo-rh.com:3478
    username: leopardo
    password: ${TURN_PASSWORD} # Variable d'environnement

# ■■ Logs
logLevel: info
logDestinations: [file]
logFile: /var/log/mediamtx/mediamtx.log

# ■■ Paths (configuration par caméra)
paths:
  all: # Règle générale pour toutes les caméras
    source: publisher # La source est un publisher externe (caméra en PUSH)
    # OU pour mode PULL (MediaMTX se connecte à la cam) :
    # source: rtsp://${RTSP_URL} # configuré dynamiquement
    maxReaders: 10 # Max connexions simultanées par flux
    readTimeout: 30s
    writeTimeout: 30s
```

Important : en mode PULL (recommandé pour Phase 1 Prime), Laravel transmet l'URL RTSP chiffrée à MediaMTX via l'API de gestion MediaMTX (POST /v3/config/paths/add). Ainsi MediaMTX maintient une connexion permanente à la caméra, et le stream est disponible immédiatement quand un client s'y connecte.

Script de démarrage MediaMTX (systemd service) :

```
# /etc/systemd/system/mediamtx.service
[Unit]
Description=MediaMTX Video Proxy – Leopardo RH
After=network.target

[Service]
ExecStart=/usr/local/bin/mediamtx /etc/mediamtx/mediamtx.yml
Restart=always
RestartSec=5s
Environment=TURN_PASSWORD=your_secret_here
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

8

PLAN D'IMPLÉMENTATION — TOUTES LES TÂCHES

60+ tâches détaillées avec priorités, efforts et dépendances

Convention : Effort en jours-développeur (8h). Priorité Critique = bloquant. Haute = nécessaire pour la feature. Moyenne = qualité/polish. Les sprints sont ordonnés par dépendance logique.

SPRINT A — Infrastructure & Base de données (Semaine 1)

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-01	Migrations PostgreSQL	3 migrations Laravel : create_cameras_table, create_camera_access_tokens_table, create_camera_permissions_table. Tous les champs, contraintes FK, CHECK. Rollback testé.	0,5 j	Critique	—
CAM-02	Index PostgreSQL	Créer tous les index listés en section 4.5. EXPLAIN ANALYZE sur la requête de validation token pour confirmer < 1ms.	0,25 j	Critique	CAM-01
CAM-03	Seeder mise à jour plans	UpdatePlansFeaturesCamerasSeeder : ajouter cameras:true, max_cameras:4 dans Business et cameras:true, max_cameras:null dans Enterprise. Idempotent (peut être rejoué).	0,25 j	Critique	CAM-01
CAM-04	Installation MediaMTX sur VPS	Télécharger binary mediamtx v1.x, configurer mediamtx.yml (auth webhook, TLS, WebRTC). Tester avec une caméra réelle. Configurer systemd service (restart=always).	1 j	Critique	—
CAM-05	Déploiement coturn (TURN server)	Installer coturn sur le VPS. Configurer turnserver.conf (port 3478 UDP, realm, credentials). Tester avec un client mobile derrière NAT.	0,5 j	Haute	CAM-04
CAM-06	Variables d'environnement	Ajouter dans .env : MEDIAMTX_BASE_URL, MEDIAMTX_API_URL, MEDIAMTX_INTERNAL_SECRET, CAMERA_STREAM_TOKEN_TTL=14400, CAMERA_RTSP_CIPHER_KEY, TURN_URL, TURN_USER, TURN_PASS. Mettre à jour .env.example et documentation.	0,25 j	Critique	CAM-04

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-07	Modèle Camera (Eloquent)	app/Models/Camera.php : chiffrement/déchiffrement rtsp_url via Crypt::encryptString(), SoftDeletes, scope company (Global Scope), relations hasMany(CameraAccessToken), hasMany(CameraPermission), belongsTo(Employee, 'created_by').	0,5 j	Critique	CAM-01
CAM-08	Modèle CameraAccessToken	app/Models/CameraAccessToken.php : scope actifs (is_revoked=false AND expires_at>now()), méthode statique generate(camera_id, label, ttl), méthode updateLastUsed(), relation belongsTo(Camera).	0,25 j	Critique	CAM-01
CAM-09	Modèle CameraPermission	app/Models/CameraPermission.php : contrainte unique (camera_id, employee_id), scopes hasViewAccess(\$employee), hasShareAccess(\$employee).	0,25 j	Haute	CAM-01
CAM-10	StreamTokenService	app/Services/StreamTokenService.php : generate(Camera \$cam, Company \$co) → JWT signé HMAC-SHA256 avec {cam_id, company_id, exp}. Méthode verify(string \$token) → array false. TTL configurable via CAMERA_STREAM_TOKEN_TTL.	0,75 j	Critique	CAM-07
CAM-11	CameraPolicy	app/Policies/CameraPolicy.php : viewAny (manager + plan.cameras), view (même cam company), create (manager:principal + max_cameras check), update/delete (manager:principal), share (manager).	0,5 j	Critique	CAM-07 CAM-10
CAM-12	Middleware CheckCameraSubscription	Étendre CheckSubscription existant pour vérifier features.cameras=true. Si false : retourner 403 avec message d'upgrade. Enregistrer dans Kernel.php sous alias 'camera.subscription'.	0,25 j	Critique	CAM-11

SPRINT B — API Backend Laravel (Semaine 2)

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-13	CameraController (CRUD complet)	Api/V1/CameraController : index() génère stream_tokens via StreamTokenService pour chaque cam, store() valide + chiffre rtsp_url + vérifie max_cameras, update(), destroy() soft-delete. Utilise CameraPolicy.	1,5 j	Critique	CAM-07 CAM-10 CAM-11 CAM-12
CAM-14	CameraAccessTokenController	Api/V1/CameraAccessTokenController : index() liste tokens non-révoqués triés par expires_at, store() génère token + envoie éventuellement email au tiers, destroy() is_revoked=true. Max 30 tokens actifs par caméra.	1 j	Haute	CAM-08 CAM-13
CAM-15	InternalCameraTokenController	Contrôleur dédié hors middleware Sanctum, protégé par MEDIAMTX_INTERNAL_SECRET. Vérifie stream_token JWT OU access_token tiers. Met à jour last_used_at et use_count. Logs chaque vérification.	0,75 j	Critique	CAM-10 CAM-08
CAM-16	Form Requests (validation)	CreateCameraRequest : name required string max:100, rtsp_url required regex:^(rtsp://, location nullable string max:200, sort_order nullable integer min:0. CreateAccessTokenRequest : label nullable, expires_in enum[3600,21600,86400,604800,2592000], granted_to_email nullable email.	0,5 j	Haute	CAM-13
CAM-17	Routes API V1	routes/api.php : groupe middleware [sanctum, tenant, camera.subscription] pour /cameras. Route séparée sans Sanctum pour /internal/camera-token/verify. Route publique /view/cam. Respecter convention nommage api.v1.cameras.*.	0,25 j	Critique	CAM-13 CAM-14 CAM-15
CAM-18	Endpoint test-rtsp	POST /api/v1/cameras/test-rtsp : accepte rtsp_url, lance ffprobe -v quiet -show_streams -of json {url} avec timeout Process 5s. Retourne {success:bool, codec:string, resolution:string, error:string}. Nécessite ffprobe installé sur le serveur.	0,5 j	Moyenne	CAM-13
CAM-19	Synchronisation MediaMTX (CameraObserver)	Observer Eloquent sur Camera : après create() → appel API MediaMTX POST /v3/config/paths/add pour configurer le RTSP pull. Après delete() → DELETE /v3/config/paths/remove. Gestion erreur gracieuse (log sans bloquer).	0,75 j	Haute	CAM-07 CAM-04

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-20	CameraAccessLog Controller	GET /api/v1/cameras/{id}/access-logs : retourne historique paginé des vérifications de tokens (depuis les logs ou table camera_access_logs optionnelle). Utile pour audit accès tiers.	0,5 j	Moyenne	CAM-15
CAM-21	API Resources (Transformers)	CameraResource, CameraCollection : transforment les modèles en JSON propre. Inclure stream_token et stream_url générés dynamiquement dans CameraResource. CameraAccessTokenResource : masquer le token brut après création (afficher seulement à la création).	0,5 j	Haute	CAM-13
CAM-22	Tests Pest — Sprint B	Tests : create cam (succès + rtsp invalide + plan insuffisant + max_cameras dépassé), list (stream_token présent + expiré), delete (soft delete vérifié), test-rtsp (success + timeout), verify-token (valide + expiré + révoqué + autre company). Coverage > 85%.	2 j	Haute	CAM-13 CAM-14 CAM-15

SPRINT C — Dashboard Web Blade (Semaine 3)

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-23	Routes Web Blade	routes/web.php : groupe /cameras avec middleware [auth:web, manager, camera.subscription]. 6 routes : index, create, store, show, edit, update. Route publique /view/cam pour accès tiers. Convention nommage manager.cameras.*.	0,25 j	Haute	CAM-17
CAM-24	WebCameraController (Blade)	Web/WebCameraController : index() inject liste + plan_info, create/edit() formulaires, store/update() appel logique CameraController réutilisée, show() prépare stream_token pour la vue. Redirect avec flash message.	0,75 j	Haute	CAM-13 CAM-23
CAM-25	Vue index.blade.php (liste caméras)	Grille responsive 2-4 colonnes de cards caméras. Chaque card : thumbnail (ou placeholder animé), nom, badge actif/inactif, localisation, boutons Voir / Gérer. Section plan_info : X/Y caméras utilisées, bouton upgrade si limite atteinte.	1 j	Haute	CAM-24
CAM-26	Vue create-edit.blade.php	Formulaire : champ name, rtsp_url (input password avec toggle show/hide), location, sort_order, is_active toggle. Bouton 'Tester la connexion' AJAX (→ test-rtsp). Affichage résultat test : codec, résolution ou message d'erreur.	0,75 j	Haute	CAM-24
CAM-27	Composant Alpine.js WebRTCPlayer	x-data='webrtcPlayer({streamUrl, streamToken, turnConfig})' : encapsule RTCPeerConnection + fetch SDP offer/answer. États : connecting, playing, error, reconnecting. Reconnexion auto après 5s si déconnecté. Affiche statut connexion en overlay.	2 j	Critique	CAM-24
CAM-28	Vue show.blade.php (vue caméra unique)	Lecteur plein-largeur avec WebRTCPlayer. Sidebar info : nom, localisation, statut flux, dernier accès. Bouton plein-écran (Fullscreen API). Bouton refresh token manuel.	0,75 j	Haute	CAM-27
CAM-29	Vue grille multi-caméras (dashboard cam)	Affichage 1/2/4 flux simultanés en mosaïque (layout grid CSS). Sélecteur de layout en haut à droite. Clic sur une cellule → agrandissement. Les flux non-visibles sont mis en pause (économie bande passante).	1,5 j	Haute	CAM-27

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-30	Vue gestion accès tiers	Liste tokens actifs : label, email, créé par, expiration, compteur d'utilisation, badge Actif/Expiré/Révoqué. Actions : Copier le lien, Envoyer par email (optionnel), Révoquer avec confirmation. Formulaire nouveau token avec sélecteur durée (radio ou select).	1 j	Haute	CAM-14 CAM-24
CAM-31	Page accès tiers /view/cam	Route publique : vérifie access_token, affiche flux WebRTC sans menu ni navigation Leopard RH. Header minimal avec nom caméra + logo. Messages clairs : Accès expiré, Accès révoqué, Caméra inactive. Pas de lien vers le dashboard.	0,75 j	Haute	CAM-15 CAM-27
CAM-32	Email notification accès tiers (opt.)	CameraAccessGrantedMail : envoyé au granted_to_email si fourni lors de la création du token. Contient le lien, la durée, le nom du manager, et un texte légal. Utilise la queue Laravel existante.	0,5 j	Moyenne	CAM-14
CAM-33	Mention légale surveillance (onboarding)	Modal obligatoire à l'ajout de la 1re caméra. Le manager doit cocher : (1) J'ai informé mes employés de la présence de caméras, (2) Les caméras filment uniquement les espaces communs. Confirmation stockée dans companies.metadata.	0,5 j	Critique	CAM-26
CAM-34	Tests E2E Blade (Dusk ou Pest Browser)	Tests : ajout caméra (formulaire + validation), test-rtsp (mock réponse), liste caméras (plan limit affiché), génération token tiers (lien copié), révocation token, page /view/cam (accès valide + expiré).	1 j	Haute	CAM-25 CAM-26 CAM-30 CAM-31

SPRINT D — Application Flutter (Semaines 4-5)

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-35	Dépendances Flutter	pubspec.yaml : ajouter flutter_webrtc: ^0.9.x. Configurer permissions iOS : NSCameraUsageDescription, NSMicrophoneUsageDescription (pour future audio). Android : INTERNET permission (déjà présente). Vérifier conflits avec dépendances existantes.	0,5 j	Critique	—
CAM-36	CameraModel (Dart)	lib/models/camera_model.dart : fromJson/toJson, champs id/name/location/isActive/streamUrl/streamToken/tokenExpiresAt/thumbnailUrl. Méthode isTokenExpiringSoon() (true si < 30min). Méthode isTokenValid().	0,25 j	Critique	CAM-35
CAM-37	CameraRepository	lib/repositories/camera_repository.dart : getCameras() → Future<, refreshStreamToken(id) → Future, createAccessToken(id, {label, expiresIn}) → Future, revokeAccessToken(id, tokenId). Réutilise DioClient + ApiService existants.	0,75 j	Critique	CAM-36
CAM-38	CameraBloc / CameraNotifier	Gestion d'état (BLoC ou Riverpod selon pattern existant). États : CameraInitial, CameraLoading, CameraLoaded(cameras), CameraError(message), CameraStreamActive(cameraId). Events : LoadCameras, RefreshToken(id), ToggleCamera(id). Timer auto-refresh 30min.	1 j	Critique	CAM-37
CAM-39	WebRTCPlayerWidget	Widget réutilisable : RTCVideoRenderer + RTCPeerConnection. Méthode initConnection(streamUrl, token, turnConfig). Gestion états : connecting (spinner) / playing (video) / error (message + retry) / reconnecting (overlay + backoff exponentiel 5s→10s→30s). Dispose propre.	2,5 j	Critique	CAM-35 CAM-38
CAM-40	Écran CameraListScreen	Grille scrollable de CameraCard. CameraCard : thumbnail (CachedNetworkImage ou placeholder animé), nom, badge statut, localisation. Tap → CameraViewScreen. FAB d'accès si manager:principal. Badge plan : '2/4 caméras'. Masqué si plan insuffisant (remplacé par upsell card).	1 j	Critique	CAM-36 CAM-38

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-41	Navigation vers le module caméras	Ajouter icône caméra dans la BottomNavigationBar manager (entre Employés et Dashboard). Visible seulement si plan.features.cameras=true (vérification dans HomeBloc). Badge numérique si N caméras actives.	0,5 j	Critique	CAM-40
CAM-42	Écran CameraViewScreen (vue unique)	Lecteur plein-écran avec WebRTCPlayerWidget. AppBar : nom caméra, bouton plein-écran, bouton partage (→ bottom sheet). Overlay info en bas : localisation, statut, qualité réseau (RTCStatsReport). Rotation écran supportée.	1 j	Haute	CAM-39 CAM-40
CAM-43	Écran multi-caméras (GridView)	Affichage 1/2/4 flux en grille. Boutons sélecteur layout (icônes 1x1, 2x1, 2x2). Tap sur cellule → plein écran de cette caméra. Flux non-visibles arrêtés (dispose) pour économiser batterie et bande passante. Bouton retour → liste.	1,5 j	Haute	CAM-39 CAM-42
CAM-44	Gestion auto-refresh token Flutter	CameraBloc lance Timer.periodic toutes les 30 secondes, vérifie isTokenExpiringSoon() sur chaque caméra active. Si vrai → refreshStreamToken() en arrière-plan. Si refresh échoue → afficher banner 'Session expirée' avec bouton retry. Ne pas couper le flux pendant le refresh.	0,75 j	Haute	CAM-38 CAM-39
CAM-45	Bottom sheet partage accès tiers	Sheet ouvert depuis CameraViewScreen bouton partage : liste tokens actifs + bouton 'Nouveau lien'. Formulaire inline : sélecteur durée (chips: 1h, 6h, 24h, 7j), champ label, champ email optionnel. Bouton Copier lien + Partager (Share.share). Bouton révoquer par token.	1 j	Haute	CAM-37 CAM-42
CAM-46	Indicateur qualité réseau	Analyser RTCStatsReport toutes les 5s : extraire bytesReceived, packetsLost. Calculer qualité : Excellente / Bonne / Moyenne / Faible. Afficher badge coloré (vert/jaune/rouge) en overlay sur le flux. En cas de mauvaise qualité : proposer réduction résolution (Phase 2).	0,5 j	Moyenne	CAM-39
CAM-47	Tests widget Flutter	Tests : CameraCard (rendu avec/sans thumbnail, badge plan limit), CameraListScreen (état loading/loaded/error/vide), CameraBloc (transition états, auto-refresh, gestion erreur), WebRTCPlayerWidget (mock connexion, états). Utiliser mockito pour les repos.	1,5 j	Haute	CAM-38 CAM-40

SPRINT E — Tests, Sécurité & Déploiement (Semaine 6)

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-48	Tests E2E complet (scénario golden path)	Scénario : manager crée cam → voir liste → ouvre flux → stream_token refresh auto → génère lien tiers → tiers ouvre lien dans navigateur → voit flux → manager révoque → tiers voit 'Accès révoqué' dans 30s. Documenter avec screenshots.	1 j	Haute	CAM-22 CAM-34 CAM-47
CAM-49	Audit sécurité — isolation multitenancy	Tests : accès caméra company B avec token company A (doit échouer), replay stream_token expiré (doit échouer), accès /internal/verify sans MEDIAMTX_INTERNAL_SECRET (doit échouer 403), bruteforce access_token (rate limiting). Documenter les tests dans SECURITY.md.	0,75 j	Critique	CAM-15 CAM-22
CAM-50	Test de charge MediaMTX	Simuler 20 flux simultanés avec ffplay ou script k6. Mesurer : latence p50/p95/p99, CPU VPS, mémoire. Objectif : p95 < 500ms, CPU < 50%. Ajuster maxReaders et writeQueueSize si nécessaire. Publier rapport de test.	0,5 j	Haute	CAM-04
CAM-51	Monitoring et alerting	Ajouter healthcheck MediaMTX dans Uptime Robot (GET https://proxy.leopardo-rh.com/v3/config). Alerting email/Slack si down. Dashboard super-admin : afficher nombre de flux actifs (API MediaMTX GET /v3/paths/list). Widget métriques.	0,5 j	Haute	CAM-04
CAM-52	Rate limiting endpoints caméras	Ajouter throttle(60, 1) sur endpoints publics (/view/cam, /verify). Throttle(30, 1) sur test-rtsp (prévenir abus). Throttle(10, 1) sur génération access_token. Via middleware RateLimiter::for() dans RouteServiceProvider.	0,25 j	Haute	CAM-17
CAM-53	Documentation technique MediaMTX	Rédiger mediamtx.yml commenté (fourni en section 7). Guide installation VPS step-by-step. Guide configuration routeur (redirection port RTSP) pour les 5 marques principales. Intégrer dans le RUNBOOK_LOCAL_TESTS.md existant du projet.	1 j	Haute	CAM-04
CAM-54	Guide utilisateur manager (in-app)	Écran d'aide in-app (FAQ) accessible depuis la liste caméras. 5 questions : Comment ajouter une caméra, Que faire si le flux ne s'affiche pas, Comment partager un accès, Quelles caméras sont compatibles, Combien de caméras puis-je ajouter.	0,5 j	Moyenne	CAM-40

ID	Tâche	Description	Effort	Priorité	Dépendances
CAM-55	Mention légale et conformité RGPD	CAM-33 (modal onboarding) + journal de consentement dans companies.metadata. Politique de conservation : les access_logs sont supprimés après 90 jours (job quotidien). Doc RGPD pour les marchés FR et TN dans le dossier légal du projet.	0,5 j	Critique	CAM-33
CAM-56	CI/CD — pipeline tests caméras	Ajouter les tests Pest CAM-22 dans le workflow GitHub Actions tests.yml existant. Job séparé 'camera-tests' avec base de données de test. Ajouter étape linting mediamtx.yml dans le pipeline. Badge CI dans README.	0,25 j	Haute	CAM-22
CAM-57	Mise à jour PILOTAGE.md et CHANGELOG	Ajouter Phase 1 Prime dans PILOTAGE.md avec scope caméras, endpoints ajoutés, date cible. Retirer le module caméras de la liste EXCLUDED. Ajouter entrée CHANGELOG avec toutes les fonctionnalités livrées.	0,25 j	Haute	Tout

9

TESTS — STRATÉGIE COMPLÈTE

Coverage cible, cas de test critiques, automatisation

9.1 — Matrice de couverture des tests

Couche	Framework	Coverage cible	Cas critiques
API Backend (Pest PHP)	Pest + PHPUnit	> 85%	Auth multi-tenancy, plan limits, token lifecycle
Politiques (Pest)	Pest + RefreshDB	100% des politiques	Tous les rôles sur toutes les actions
Endpoint /verify (Pest)	Pest + Mock	100%	Token valide/expiré/révoqué/autre-company
Dashboard Blade (Dusk)	Laravel Dusk	Flux principaux	Formulaire cam, génération lien, révocation
Flutter Widgets	flutter_test	> 70%	États BLoC, card rendu, reconnexion auto
E2E Golden Path	Manuel + script	1 scénario complet	Création → stream → partage → révocation
Sécurité (manuel)	Tests manuels	Tous les vecteurs	Isolation tenant, token replay, bruteforce
Charge (k6)	k6 load test	20 flux simultanés	Latence p95 < 500ms, CPU < 50%

9.2 — Cas de test Pest critiques

```
// tests/Feature/Camera/CameraAccessTest.php

it('refuses camera access to wrong company', function () {
    $companyA = Company::factory()->withCamerasPlan()->create();
    $companyB = Company::factory()->withCamerasPlan()->create();
    $camera = Camera::factory()->for($companyA)->create();
    $managerB = Employee::factory()->manager()->for($companyB)->create();

    actingAs($managerB)
        ->getJson("/api/v1/cameras/{$camera->id}")
        ->assertStatus(404); // Pas 403 – ne pas révéler l'existence
});

it('blocks access when plan does not include cameras', function () {
    $manager = Employee::factory()->manager()->forCompany(['plan' => 'starter'])->create();
    actingAs($manager)
        ->getJson('/api/v1/cameras')
        ->assertStatus(403)
        ->assertJsonFragment(['upgrade' => true]);
});

it('generates fresh stream_token with correct TTL', function () {
    $camera = Camera::factory()->forAuthenticatedManager()->create();
    $response = actingAs($this->manager)->getJson('/api/v1/cameras');
    $token = $response->json('data.0.stream_token');
    $payload = (new StreamTokenService)->verify($token);
    expect($payload['cam_id']->toBe($camera->id)
        ->and($payload['exp']->toBeGreaterThan(now()->timestamp + 3600));
});
```

10

DÉPLOIEMENT ET DEVOPS

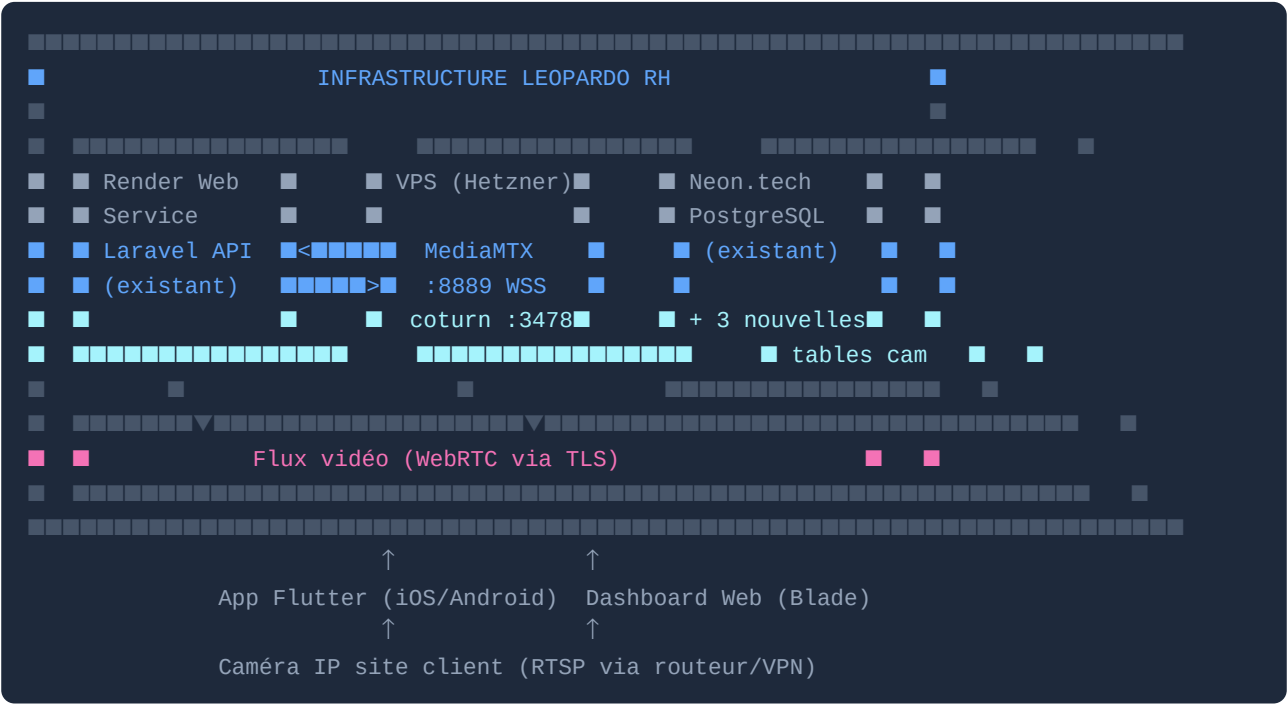
Checklist de mise en production et runbook

10.1 — Checklist déploiement

#	Action	Responsable	Vérification
1	MediaMTX binaire déployé + service systemd actif	Dev Ops	GET https://proxy.leopardo-rh.com/v3/config → 200
2	coturn déployé + port UDP 3478 ouvert dans firewall	Dev Ops	Test WebRTC depuis mobile 4G
3	Variables .env production mises à jour	Dev	php artisan config:cache sans erreur
4	Migrations lancées en production	Dev	php artisan migrate --force
5	Seeder plans lancé (cameras features)	Dev	Vérifier via super-admin dashboard
6	TLS cert proxy.leopardo-rh.com valide	Dev Ops	SSL Labs test : grade A
7	Webhook MediaMTX → Laravel accessible	Dev	Test manuel curl depuis le VPS MediaMTX
8	Tests Pest complets passent en staging	Dev	CI badge vert
9	ffprobe installé sur le serveur Laravel	Dev Ops	which ffprobe retourne un path
10	Monitoring Uptime Robot configuré	Dev Ops	Alert email/Slack reçu en cas de down
11	Mention légale RGPD validée par équipe légale	Produit	Texte validé + stocké dans consentement
12	Guide utilisateur in-app déployé	Dev	Visible dans l'app prod
13	Test E2E golden path validé en staging	QA	Scénario documenté passé
14	Rollback plan documenté	Dev Ops	Procédure testée

10.2 — Architecture de déploiement cible

Schéma des composants déployés en production :



11

DOCUMENTATION & LÉGAL

Obligations légales, conformité RGPD, guide client

11.1 — Obligations légales par pays

Pays	Texte applicable	Obligation principale	Action requise dans l'app
France	RGPD + Code du Travail Art. L1222-4	Information préalable des employés obligatoire. Déclaration CNIL optionnelle (RGPD).	Modal obligatoire + case à cocher manager
Algérie	Loi 18-07 Protection données personnelles	Consentement éclairé des personnes filmées.	Modal obligatoire
Maroc	Loi 09-08	Déclaration à la CNDP pour traitement automatisé.	Modal + lien documentation CNDP
Tunisie	Loi organique 63-2004	Autorisation INPDP requise pour surveillance.	Modal + mise en garde + lien INPDP
Turquie	KVKK (RGPD turc)	Information obligatoire + enregistrement VERBİS si > 50 employés.	Modal obligatoire

11.2 — Texte du consentement obligatoire (modal)

Ce texte doit être affiché verbatim lors de la configuration de la première caméra, dans la langue de l'interface :

En activant la surveillance vidéo dans Leopardo RH, vous confirmez avoir : (1) informé vos employés de la présence et du but des caméras de surveillance, (2) veillé à ce que les caméras filment uniquement des espaces de travail communs et non des espaces privés (vestiaires, sanitaires, espaces de repos), (3) respecté les obligations légales en vigueur dans votre pays concernant la surveillance des employés. Leopardo RH ne peut être tenu responsable du non-respect de ces obligations.

12

ROADMAP POST PHASE 1 PRIME

Évolutions planifiées — à documenter et développer après la Phase 1P

Ces features sont spécifiées ici pour guider les choix d'architecture en Phase 1 Prime (ex: prévoir les colonnes pour le recording, les hooks pour la détection de mouvement). Elles ne sont pas à implémenter maintenant.

PHASE 2 — Enregistrement vidéo et stockage (S3)

ID	Tâche	Description	Effort	Priorité	Dépendances
P2-01	Recording continu par caméra	MediaMTX supporte natif le recording RTSP → MP4 via recorder. Configurer chemin de sortie par camera_id. Rotation automatique des segments (toutes les 15 min). Transférer segments vers S3 via job Laravel asynchrone.	3 j	Haute	CAM-04
P2-02	Table camera_recordings	Colonnes : camera_id, started_at, ended_at, file_path (S3), file_size, duration_sec, thumbnail_path. Index sur (camera_id, started_at). Rétention configurable par plan (7j Starter, 30j Business, 90j Enterprise).	0,5 j	Haute	P2-01
P2-03	API et UI replay vidéo	GET /api/v1/cameras/{id}/recordings (liste par date). GET /api/v1/cameras/{id}/recordings/{rid}/url (URL S3 présignée TTL=15min). Vue replay dans le dashboard avec timeline calendaire. Player vidéo HTML5 avec contrôle de vitesse.	3 j	Haute	P2-02
P2-04	Téléchargement segments	Bouton Télécharger sur un segment : génère URL S3 présignée. Limite par plan (ex: 3 téléchargements/jour en Business). Log de chaque téléchargement pour audit.	1 j	Moyenne	P2-03
P2-05	Purge automatique et rétention	Job quotidien PurgeOldRecordingsJob : supprime de S3 et de la table les recordings dépassant la rétention du plan. Notification au manager 3 jours avant purge.	0,75 j	Haute	P2-02
P2-06	Modèle tarifaire recording	Ajouter dans plans.features : recording_enabled, recording_retention_days (7/30/90). Afficher consommation S3 en Go dans le dashboard super-admin. Facturation sur consommation Phase 3+.	0,5 j	Haute	P2-02

PHASE 3 — Détection de mouvement et alertes IA

ID	Tâche	Description	Effort	Priorité	Dépendances
P3-01	Pipeline de détection de mouvement	Analyser les frames vidéo via un service Python dédié (OpenCV ou PyTorch MobileNet). MediaMTX peut exposer les frames via HLS ou RTSP pour analyse. Service Python abonne aux flux et déclenche une alerte via webhook Laravel.	5 j	Haute	CAM-04
P3-02	Table camera_motion_events	camera_id, detected_at, confidence (0-1), zone (null ou polygon JSON), snapshot_path (S3). Index sur (camera_id, detected_at). Rétention 30 jours.	0,5 j	Haute	P3-01
P3-03	Configuration zones de détection	Interface drag-and-drop dans le dashboard : dessiner des polygones sur la preview statique de la caméra pour définir les zones actives. Stocké en JSONB dans cameras.metadata.	3 j	Haute	P3-01
P3-04	Alertes push sur détection	Notification push Flutter (Firebase/FCM ou APNs) lors d'une détection dans une zone définie. Throttling : max 1 alerte par caméra par tranche de 5 minutes (éviter le spam). Snapshot joint à la notification.	2 j	Haute	P3-02
P3-05	Plages horaires de détection	Configuration horaires d'activation de la détection par caméra (ex: 19h-7h uniquement). Stocké dans cameras.metadata. Job Laravel évalue les plages avant de déclencher l'alerte.	1 j	Haute	P3-03
P3-06	Historique des détections (UI)	Timeline des événements de mouvement dans le dashboard. Filtres : caméra, date, confiance minimale. Clic sur un événement → snapshot + lien vers le recording correspondant (si Phase 2 actif).	2 j	Haute	P3-02
P3-07	Détection de présence hors-horaires	Alerte spéciale si mouvement détecté en dehors des heures de travail configurées dans les paramètres de la company. Couplé avec le module absences existant pour croiser les données.	1,5 j	Moyenne	P3-04
P3-08	Mode Privacy : masquage zones	Floutage en temps réel de zones définies dans les flux (ex: fenêtres vers l'extérieur, zone privée). Implémenté via OpenCV blur polygon. Respecte les obligations légales RGPD.	3 j	Moyenne	P3-01

PHASE 4 — Analytics vidéo avancé

ID	Tâche	Description	Effort	Priorité	Dépendances
P4-01	Comptage de personnes (People Counting)	Modèle IA (YOLOv8 ou SSD) pour compter le nombre de personnes dans une zone à tout moment. Dashboard : affluence en temps réel, historique par heure/jour. Applicable pour commerce et accueil.	8 j	Haute	P3-01
P4-02	Heatmaps de fréquentation	Superposition de heatmap sur la preview statique montrant les zones les plus fréquentées sur une période. Export image PNG. Utilise les données de People Counting.	4 j	Moyenne	P4-01
P4-03	Détection d'équipement de protection (EPI)	Vérifier si les employés portent casque, gilet, lunettes dans les zones à risque. Alerte si EPI manquant. Applicable pour industrie et chantier.	10 j	Haute	P3-01
P4-04	Analyse comportementale (anomalies)	Détection de comportements anormaux : chute de personne, regroupement inhabituel, véhicule dans zone interdite. Via réseau de neurones LSTM sur séquences vidéo.	15 j	Haute	P3-01
P4-05	Rapport automatique de surveillance	PDF hebdomadaire automatique : résumé des alertes, graphiques d'affluence, incidents détectés. Envoyé par email au manager le lundi matin. Généré par DomPDF + job Laravel.	2 j	Moyenne	P4-01 P3-06

PHASE 5 — Intégration avec l'écosystème Leopardo RH

ID	Tâche	Description	Effort	Priorité	Dépendances
P5-01	Corrélation pointage ↔ caméra	Afficher une miniature vidéo au moment du check-in d'un employé dans le dashboard de pointage. Clic → ouvre le segment recording correspondant. Preuve visuelle du pointage.	2 j	Haute	P2-03
P5-02	Accès caméras pour rôles Dept/Superviseur	Activer la configuration camera_permissions pour les sous-rôles. UI pour assigner des caméras à un superviseur/département. Scope dynamique dans CameraPolicy.	2 j	Haute	CAM-09
P5-03	Intégration ZKTeco vidéo	Coupler les pointeuses ZKTeco biométriques (déjà dans le projet) avec les caméras de la même zone. Afficher snapshot vidéo lors d'un accès refusé sur la pointeuse.	3 j	Haute	P2-03

ID	Tâche	Description	Effort	Priorité	Dépendances
P5-04	API publique caméras (plan Enterprise)	Exposer un endpoint webhook configurable : envoyer un événement (mouvement détecté, flux actif/inactif) vers un système tiers (SIEM, système de sécurité physique). Authentification HMAC.	2 j	Moyenne	P3-01
P5-05	Mode kiosque avec surveillance	Intégrer un flux caméra dans l'écran du kiosque de pointage (vue manager uniquement, accès protégé). Le manager voit qui pointe et la zone en temps réel depuis le kiosque.	3 j	Basse	P5-01
P5-06	Support PTZ (Pan-Tilt-Zoom)	Contrôle caméra PTZ depuis l'app : boutons directionnels, zoom +/- . Via protocole ONVIF (standard PTZ). Nécessite caméra compatible ONVIF. Interface intuitive sur mobile (swipe pour pan/tilt).	5 j	Basse	CAM-39

13

RÉCAPITULATIF ET DÉFINITION OF DONE

Synthèse globale du module

13.1 — Résumé par sprint (Phase 1 Prime)

Sprint	Contenu	Durée	Tâches	Effort	Livrable
A — Infrastructure	Migrations, MediaMTX, modèles, politiques	Sem. 1	12 tâches	4,25 j	Base technique opérationnelle
B — API Backend	CRUD cameras, tokens, tests Pest	Sem. 2	10 tâches	8,25 j	API complète et testée
C — Dashboard Web	Blade : vues, formulaires, player WebRTC, accès tiers	Sem. 3	12 tâches	9,25 j	Interface web fonctionnelle
D — App Flutter	Écrans, WebRTCPlayer, multi-cam, tokens	Sem. 4-5	13 tâches	11,75 j	App mobile complète
E — Tests & Deploy	E2E, sécurité, charge, docs, CI/CD, légal	Sem. 6	10 tâches	5,75 j	Module prêt pour beta
TOTAL Phase 1P	Module caméras complet (streaming + accès tiers)	6 semaines	57 tâches	39,25 j	Feature production-ready

13.2 — Résumé roadmap post-Phase 1P

Phase	Contenu	Tâches	Effort estimé	Valeur ajoutée
Phase 2	Enregistrement S3, replay, purge automatique	6 tâches	~8,75 j	Preuve légale, audit
Phase 3	Détection mouvement, alertes push, zones, privacy	8 tâches	~18 j	Sécurité active, alerting
Phase 4	People counting, heatmaps, EPI, comportemental, rapports	5 tâches	~39 j	Analytics site, conformité sécurité
Phase 5	Intégration RH, ZKTeco, API pub, PTZ, kiosque	6 tâches	~17 j	Plateforme site complète
TOTAL Roadmap	Module caméras complet (streaming + IA + analytics)	25 tâches	~82,75 j	Différenciation marché maximale

13.3 — Définition of Done — Phase 1 Prime

ID	Catégorie	Critère
CAM-DOD-01	FONCTIONNEL	Un manager Plan Business ou Entreprise peut ajouter une caméra IP avec son URL RTSP depuis le dashboard web.
CAM-DOD-02	FONCTIONNEL	Le manager peut visualiser le flux vidéo en direct depuis l'app Flutter (iOS et Android) en < 3 secondes.
CAM-DOD-03	FONCTIONNEL	Le manager peut visualiser jusqu'à 4 flux simultanés en vue mosaïque depuis l'app Flutter.
CAM-DOD-04	FONCTIONNEL	Le manager peut générer un lien d'accès temporaire et le partager avec un tiers sans compte. Le tiers voit le flux dans un navigateur.
CAM-DOD-05	FONCTIONNEL	La révocation d'un accès tiers coupe le flux dans les 30 secondes.
CAM-DOD-06	FONCTIONNEL	Le stream_token est automatiquement rafraîchi sans interruption du flux avant son expiration (TTL=4h).
CAM-DOD-07	SÉCURITÉ	L'isolation multitenancy est vérifiée : aucune caméra d'une entreprise n'est visible depuis le compte d'une autre entreprise.
CAM-DOD-08	SÉCURITÉ	Un stream_token expiré ou révoqué est rejeté par MediaMTX en < 100ms.
CAM-DOD-09	SÉCURITÉ	Les URLs RTSP (incluant credentials) sont chiffrées en base. Jamais exposées en clair dans les logs ou les réponses API.
CAM-DOD-10	LÉGAL	La mention légale de surveillance est affichée et validée par le manager avant la création de la première caméra.
CAM-DOD-11	PERFORMANCE	MediaMTX supporte 20 flux simultanés avec latence p95 < 500ms sur WiFi standard.
CAM-DOD-12	QUALITÉ	Coverage tests Pest > 85%. Tous les cas critiques de sécurité testés automatiquement.
CAM-DOD-13	COMPATIBILITÉ	Le module fonctionne exclusivement pour les plans Business et Entreprise. Un manager Starter voit un message d'upgrade propre.
CAM-DOD-14	MONITORING	Alerte automatique si MediaMTX est indisponible. Dashboard super-admin affiche le nombre de flux actifs.
CAM-DOD-15	DOCUMENTATION	PILOTAGE.md mis à jour. Guide utilisateur in-app disponible. Guide installation caméra (PDF client) livré.

Ce document constitue la référence complète pour l'implémentation du module caméras. Toute question d'implémentation non couverte doit être documentée et ajoutée à ce dossier avant démarrage du sprint concerné. Version 1.0 — Avril 2026 — Équipe Produit Leopardo RH.